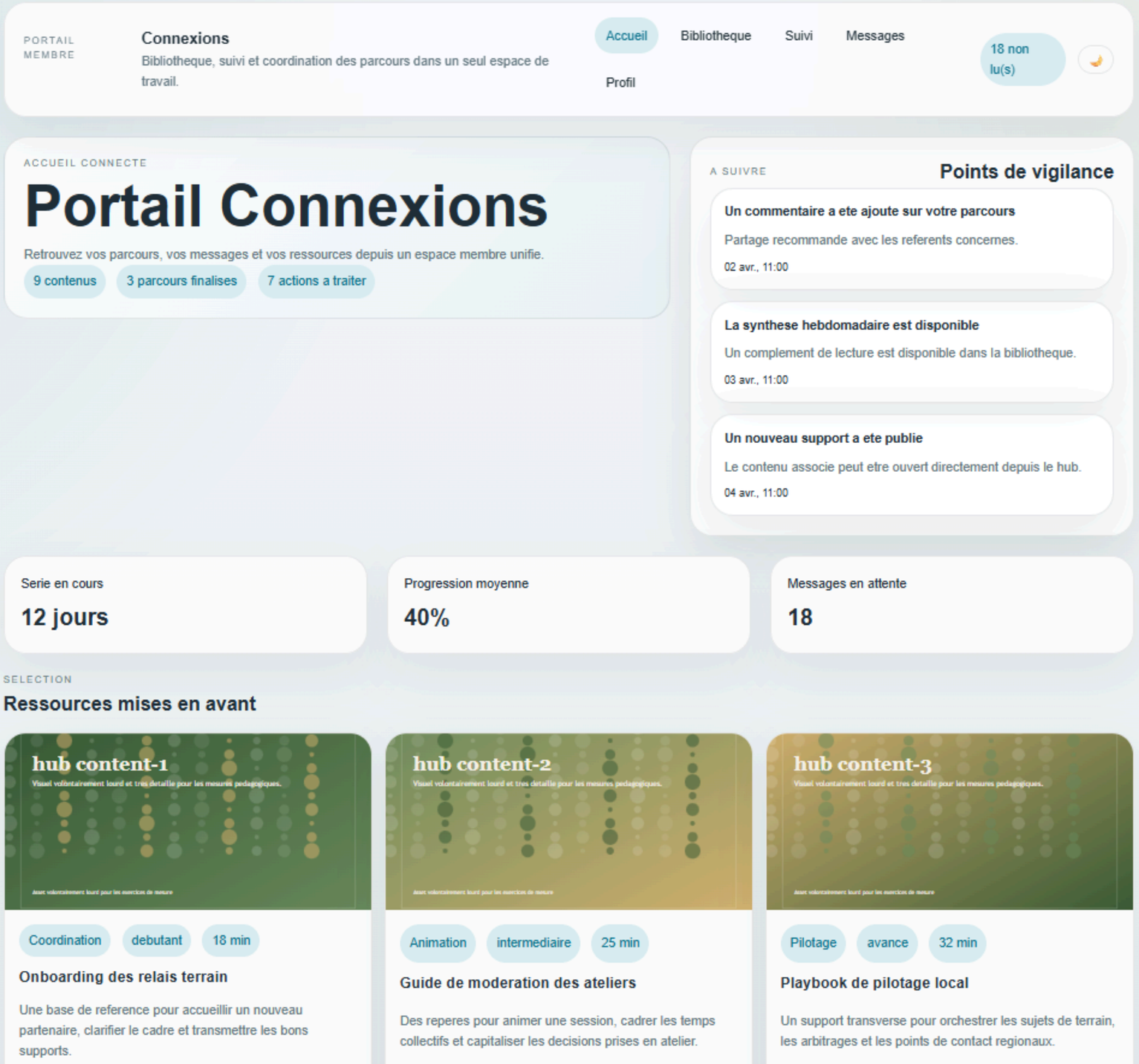


Portail Heavy-Hub

Réduire l'empreinte environnementale du portail et opportunités d'optimisation au-delà de la seule performance d'affichage.

Julie NONNON
Simplon - CDA P7 2026

Etat des lieux - Audit EcoIndex et Lighthouse

L’audit initial révèle des opportunités d'optimisation au-delà de la seule performance d'affichage.

- **Lighthouse :** performance moyenne de 89,5/100.
- **EcoIndex :** score moyen de 77,94/100 (grade B).
- **Poids moyen des pages :** 1,76 Mo.
- **Nombre moyen de requêtes :** 27,5.
- **Appels API :** 10 Fetch observés sur les pages principales au premier chargement, avec des endpoints répétés.

Etat des lieux - Audit EcoIndex et Lighthouse

Enjeu	Action envisagée
Réduire les échanges réseau	Charger uniquement les données nécessaires à chaque écran.
Alléger le frontend	Rationaliser CSS et traitements JavaScript.
Réduire les traitements inutiles	Mutualiser les transformations et limiter les rendus.
Améliorer l'expérience utilisateur	Fiabiliser les chargements et améliorer l'accessibilité.

Les performances actuelles sont satisfaisantes, mais l'audit révèle des **possibilités de sobriété importantes** dans la gestion des données et des ressources frontend.

User Stories identifiées

US1 : En tant que **Développeur Frontend**, je veux **Optimiser le CSS**, afin d'A**lléger le fichier hub.css**.

US2 : En tant que **Utilisateur**, je veux **Avoir accès à un thème sombre**, afin d'**améliorer mon confort visuel et, sur les écrans compatibles, de réduire la consommation d'énergie de l'écran.**

US3 : En tant qu'**Utilisateur**, je veux **que seules les données nécessaires à mon écran soient chargées et que les messages ne soient actualisés que lorsque c'est utile**, afin de **réduire l'attente et les échanges réseau superflus**.

US4 : En tant que **Développeur Frontend**, je veux **transformer chaque contenu une seule fois et réutiliser le résultat**, afin de **limiter le travail répété lors de l'affichage des différentes pages.**

US5 : En tant qu'**Utilisateur**, je veux **que la mise à jour d'une information ne relance pas le rendu des écrans qui ne la consultent pas**, afin de **conserver une navigation fluide**.

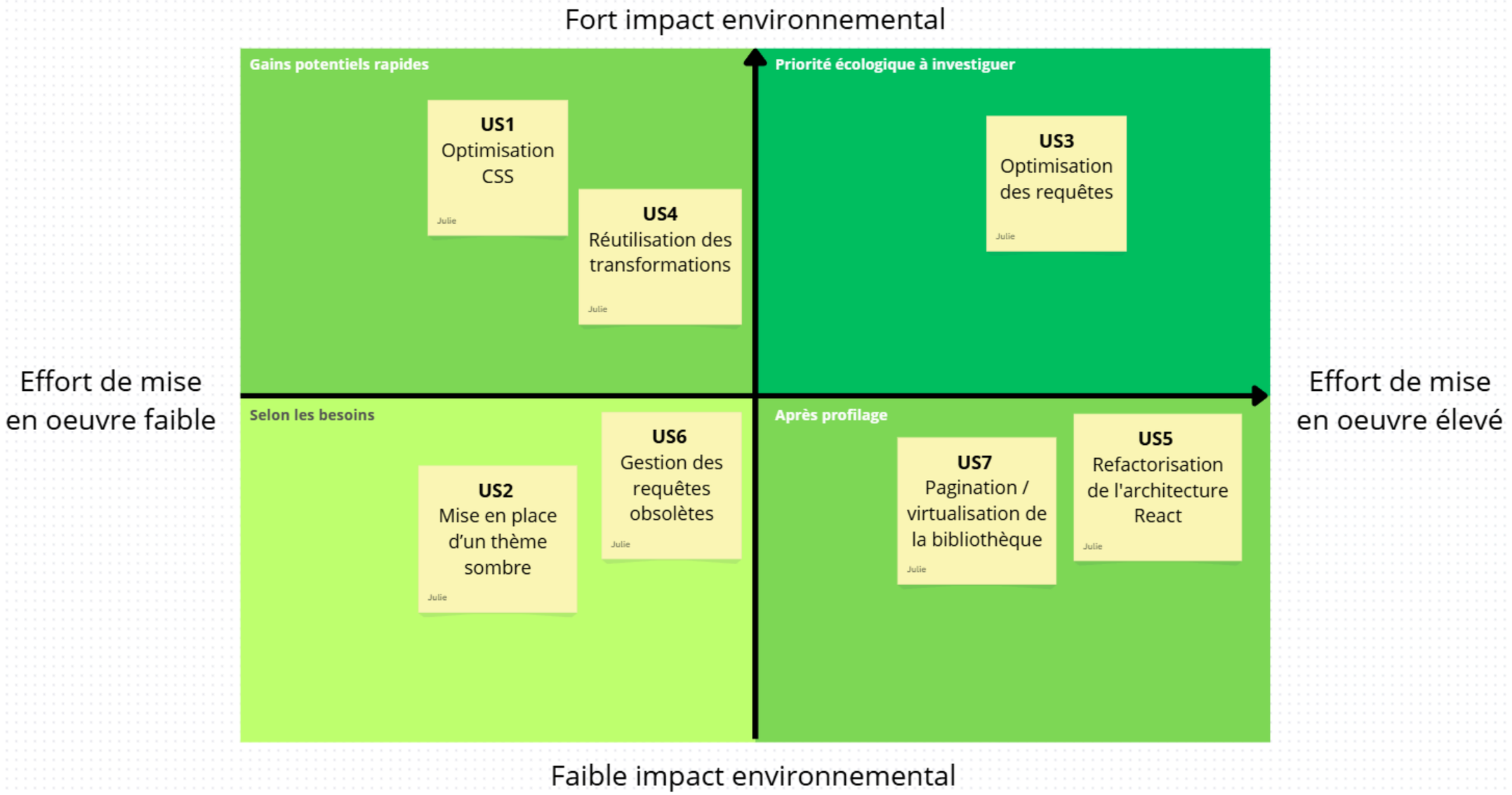
US6 : En tant qu'**Utilisateur**, je veux **voir un état de chargement, une erreur ou un contenu indisponible clairement identifié**, afin de **ne pas confondre des données anciennes avec le résultat demandé**.

US7 : En tant qu'**Utilisateur**, je veux **parcourir la bibliothèque sans attendre le rendu de toute une grande liste**, afin de **trouver rapidement une ressource**.

KPI, objectifs et bénéfices attendus

US	Action prioritaire	Constat associé	Priorité
US1	Optimiser le CSS	Ressources frontend à rationaliser	Haute
US2	Déployer le thème sombre	Amélioration du confort et de l'accessibilité	Modérée
US3	Optimiser les appels API	Appels multiples et chargements non ciblés	Très haute
US4	Mutualiser les transformations	Risque de calculs répétés	Haute
US5	Limiter les rendus React	Optimisation potentielle à profiler	Modérée
US6	Fiabiliser les requêtes	Polling notifications toutes les 5–6 secondes	Haute
US7	Optimiser les grandes listes	DOM à profiler sur des volumes représentatifs	Modérée

Matrice Impact / Effort



Matrice Impact / Effort

1. US1 et US4 — Gains potentiels rapides
Optimiser le CSS et mutualiser les transformations, en mesurant les résultats avant/après. Ce sont des chantiers ciblés si l'analyse du code confirme les duplications.

2. US3 — Priorité écologique à investiguer
Mesurer d'abord les requêtes par écran et les volumes transférés. Si les appels inutiles sont nombreux, cette US peut devenir la plus rentable.

3. US7 et US5 — Après profilage
Vérifier le coût des listes et des rendus React avant d'engager une refactorisation plus importante.

4. US6 et US2 — Selon les besoins
Traiter l'US6 pour la fiabilité et la cohérence des données. Planifier l'US2 pour l'accessibilité et le confort utilisateur, sans lui attribuer un gain énergétique garanti.

Bonnes pratiques identifiées

Bonne pratique retenue	Source du référentiel	Phase AVC concernée	Pourquoi c'est prioritaire ?	Applicabilité projet
US1 — Optimiser et rationaliser les feuilles de style CSS : supprimer les règles dupliquées, mutualiser les styles et limiter la complexité des sélecteurs.	RGESN 2024 — partie 6 « Frontend » ; GR491 — famille « Front-end ». Rattachement thématique à confirmer selon les critères détaillés retenus.	3. Développement	Réduit le poids des ressources téléchargées et peut améliorer le rendu initial. Le gain de FCP doit être mesuré, car il dépend aussi du JavaScript, du réseau et du navigateur.	Oui
US2 — Centraliser les couleurs et proposer un thème sombre accessible : réutiliser les variables CSS et éviter les ressources supplémentaires.	RGESN 2024 — partie 6 « Frontend » ; GR491 — familles « UX/UI » et « Front-end ». Rattachement thématique.	2. Conception	Améliore le confort visuel et favorise la cohérence des styles. Le bénéfice énergétique dépend notamment de la technologie de l'écran ; ce n'est pas un gain garanti sur tous les appareils.	Oui
US3 — Charger uniquement les données nécessaires et limiter les requêtes inutiles : dédupliquer les appels, réutiliser les données encore valides et arrêter les rafraîchissements hors écran.	RGESN 2024 — critères 6.1 (poids et nombre de requêtes par écran) et 6.5 (chargement des ressources utiles uniquement) ; GR491 — familles « Architecture », « Front-end » et « Back-end ».	3. Développement	Réduit les transferts réseau, le traitement côté client et les appels superflus. La mesure des requêtes par écran permet de quantifier directement les améliorations.	Oui
US4 — Transformer chaque contenu une seule fois et réutiliser le résultat : centraliser les transformations et éviter les calculs répétés.	RGESN 2024 — partie 6 « Frontend » et partie 9 « Algorithmie » ; GR491 — familles « Front-end » et « Architecture ». Rattachement thématique, sans correspondance exacte affirmée.	3. Développement	Évite les calculs redondants lorsque les transformations sont réellement coûteuses et répétées. La priorité doit être confirmée par la mesure de leur fréquence et de leur durée.	Oui
US5 — Limiter la propagation des mises à jour et isoler les états partagés : séparer les responsabilités et éviter les rendus inutiles.	RGESN 2024 — partie 6 « Frontend » ; GR491 — familles « Front-end » et « Architecture ». Rattachement thématique.	2. Conception	Une architecture d'état adaptée peut réduire le travail du navigateur et préserver la fluidité. L'intérêt environnemental dépend de la réduction effective du travail de rendu, à vérifier avec React Profiler.	Oui
US6 — Gérer les chargements, erreurs et requêtes obsolètes : annuler les appels inutiles et empêcher l'affichage de réponses périmées.	RGESN 2024 — parties 3 « Architecture » et 6 « Frontend » ; GR491 — familles « Architecture » et « Front-end ». Rattachement thématique.	3. Développement	Évite certains traitements et transferts devenus inutiles, tout en améliorant la fiabilité de l'interface. Le bénéfice écologique est secondaire par rapport au bénéfice fonctionnel et doit être mesuré.	Oui
US7 — Limiter le nombre d'éléments affichés dans les grandes listes : envisager la pagination ou la virtualisation selon les résultats du profilage.	RGESN 2024 — partie 6 « Frontend », notamment le principe de limitation des ressources inutilisées ; GR491 — famille « Front-end ». Rattachement thématique, sans assimilation à un critère spécifique de virtualisation.	2. Conception	Peut réduire le nombre de nœuds DOM, le travail de rendu et les calculs côté client. La priorité dépend du volume réel de la bibliothèque et du coût constaté au profilage.	Oui

ROADMAP
ÉCOCONCEPTION - heavy-hub

Roadmap des US sur 6 mois

	Mois 1	Mois 2	Mois 3	Mois 4	Mois 5	Mois 6
UI/UX team	Audit UI/UX + accessibilité	**US2** Mise en place d'un thème sombre	**US2** Mise en place d'un thème sombre	**US7** Pagination / virtualisation de la bibliothèque	Tests utilisateurs	Ajustements + bilan UX
Frontend team	Audit CSS, requêtes et rendus	**US1** Optimisation CSS **US4** Réutilisation des transformations	**US3** Optimisation des requêtes Optimisation des requêtes et du cache	**US6** Gestion des requêtes obsolètes	**US7** Pagination / virtualisation de la bibliothèque **US5** Refactorisation de l'architecture React	Corrections finales et stabilisation
Backend team	Etat des lieux des API	Analyse des endpoints et volumes échangés	**US3** Optimisation des requêtes Appui à la déduplication et au cache	Optimisation des API si nécessaire	Vérification des volumes et temps de réponse	Stabilisation et documentation
Testing team	Définition du protocole de test	Tests de non-régression CSS et affichage	**US3** Optimisation des requêtes Tests API et scénarii de navigation	**US6** Gestion des requêtes obsolètes Tests des états d'erreur et des requêtes annuées	**US7** Pagination / virtualisation de la bibliothèque **US5** Refactorisation de l'architecture React Tests de performances	Recette finale et comparaison avant / après
KPI & AVC	Définition des indicateurs et mesures initiales	**US1** Optimisation CSS Mesure du poids CSS et du FCP	**US3** Optimisation des requêtes Mesure des données et requêtes transférées	**US4** Réutilisation des transformations **US6** Gestion des requêtes obsolètes Mesures des transformations et requêtes obsolètes	**US7** Pagination / virtualisation de la bibliothèque **US5** Refactorisation de l'architecture React Mesures des rendus React	Bilan KPI, analyse des gains et mise à jour de l'ACV
Content team	Inventaire des contenus et volumes	Repérage des doublons et contenus inutiles	**US3** Optimisation des requêtes Vérification des données nécessaires par écrans	**US4** Réutilisation des transformations Vérification de la cohérence des contenus et libéllés	**US7** Pagination / virtualisation de la bibliothèque Contribution aux tests de la bibliothèque	Revue finale des contenus et recommandations
Pilotage projet	Cadrage, backlog et priorité	**US1** Optimisation CSS **US4** Réutilisation des transformations Suivis	**US3** Optimisation des requêtes Arbitrages	**US2** Mise en place d'un thème sombre **US6** Gestion des requêtes obsolètes Suivis	**US7** Pagination / virtualisation de la bibliothèque **US5** Refactorisation de l'architecture React Suivis	Bilan, validation et feuille de route suivante

ROADMAP
ÉCOCONCEPTION · M1

Cadrer et mesurer l’existant

DIAGNOSTIC

Poser une référence fiable avant toute optimisation

Préparation des US1, US3, US4, US5, US6 et US7

FRONTEND

Auditer CSS, appels API, transformations et rendus React.

KPI & ACV

Définir le protocole de mesure : FCP, poids CSS, requêtes, données transférées et temps de rendu.

TESTING

Préparer un jeu de données représentatif et les scénarios de référence.

PILOTAGE

Valider le périmètre, les priorités, les responsabilités et les critères de succès.

LIVRABLE **État initial documenté + tableau de bord KPI + backlog priorisé**

JALON **Jalon : mesures initiales reproductibles**

ROADMAP
ÉCOCONCEPTION · M2

Lancer les optimisations ciblées

PREMIERS GAINS

Réduire les duplications et les coûts simples à corriger

US1 — Optimiser le CSS | US4 — Réutiliser les transformations

FRONTEND

Rationaliser hub.css, regrouper les règles et centraliser les variables CSS ; mutualiser les transformations de contenu.

UI/UX

Vérifier que les styles et libellés restent identiques après les changements.

TESTING

Effectuer les tests de non-régression et comparer le poids CSS et le FCP.

KPI & ACV

Consigner les mesures avant/après et les limites de comparaison.

LIVRABLE **CSS rationalisé + transformations mutualisées + résultats de mesure**

JALON **Jalon : valider les gains sans régression fonctionnelle**

ROADMAP
ÉCOCONCEPTION · M3

Réduire les échanges réseau inutiles

EFFICACITÉ DES DONNÉES

Charger les données au plus près de leur usage

US3 — Optimiser les requêtes et le rafraîchissement des notifications

FRONTEND

Dédupliquer les appels, réutiliser un cache adapté et arrêter les rafraîchissements hors écran Notifications.

BACKEND

Analyser les endpoints, la taille des réponses et les possibilités d'optimisation si les mesures le justifient.

CONTENT

Vérifier les données réellement nécessaires à chaque écran.

TESTING / KPI

Mesurer les requêtes par écran, les doublons et le volume transféré ; tester les navigations.

LIVRABLE **Cartographie avant/après des appels API + règles de cache et de rafraîchissement documentées**

JALON **Jalon : aucun rafraîchissement inutile hors écran concerné**

ROADMAP
ÉCOCONCEPTION · M4

FIABILITÉ

Fiabiliser les parcours et les états

Éviter les réponses obsolètes et améliorer la lisibilité

US6 — Gérer les requêtes obsolètes | US2 — Thème sombre (conception / réalisation)

FRONTEND

Annuler les requêtes inutiles, empêcher les réponses tardives d'écraser les données récentes et expliciter chargement / erreur / absence.

UI/UX

Définir les couleurs du thème sombre, le contrôle de bascule et les états accessibles.

TESTING

Tester les changements rapides d'identifiant, les erreurs API et les contrastes WCAG AA.

PILOTAGE

Arbitrer les écarts et suivre les critères d'acceptation.

LIVRABLE **États de chargement et d'erreur cohérents + thème sombre validé sur les parcours couverts**

JALON **Jalon : aucune donnée obsolète affichée ; contrastes vérifiés**

ROADMAP
ÉCOCONCEPTION · M5

Optimiser les rendus coûteux

PERFORMANCE CIBLÉE

Réduire le travail du navigateur là où les mesures le justifient

US5 — Limiter les rendus React | US7 — Pagination / virtualisation

FRONTEND

Isoler les états et les composants ; implémenter une pagination ou virtualisation si le profilage confirme le besoin.

UI/UX

Valider filtres, navigation, accessibilité et expérience de la bibliothèque.

TESTING

Comparer le nombre de composants rendus, le temps de rendu et le nombre d'éléments DOM.

KPI & ACV

Documenter les gains mesurés et les compromis éventuels.

LIVRABLE **Profilage avant/après + stratégie de liste justifiée par les mesures**

JALON **Jalon : optimisations démontrées, sans mémorisation superflue**

ROADMAP
ÉCOCONCEPTION · M6

Valider les gains et pérenniser

BILAN & AMÉLIORATION CONTINUE

Consolider les preuves et préparer la suite

Validation finale des US1 à US7

TESTING

Réaliser la recette de non-régression et rejouer le protocole initial dans des conditions comparables.

KPI & ACV

Comparer les indicateurs avant/après, expliciter les écarts et mettre à jour l’analyse.

TOUTES LES ÉQUIPES

Corriger les régressions, documenter les pratiques et partager les résultats.

PILOTAGE

Valider les livrables, prioriser le backlog restant et définir le suivi dans la durée.

LIVRABLE **Bilan d’écoconception + KPI comparés + plan d’amélioration continue**

JALON **Jalon : résultats traçables et prochaines actions arbitrées**

KPI, objectifs et bénéfices attendus

Indicateur	Référence initiale	Objectif proposé au Mois 6
Score EcoIndex moyen	77,94 / 100	> 90/100
Grade EcoIndex	B	Maintenir B ou viser A si les mesures le permettent
Taille moyenne des pages	1 759 Ko	Réduction à mesurer
Nombre moyen de requêtes	27,5	Réduction à mesurer
Appels Fetch au premier chargement	10 environs	Supprimer les doublons et charger uniquement les données utiles
Polling notifications	1 appel / 5–6 s	Intervalle adapté au besoin fonctionnel
Poids CSS	10ko (données brutes sur pc)	à confirmer, idéal : -30%
FCP	1.08 s	Amélioration de 10 % sur la page représentative
LCP	1.9 s	Maintenir sous 2,5 s
DOM	145 éléments en moyenne	Réduction uniquement si le profilage le justifie

Cartographie des contraintes projet - Synthèse

Techniques

- Comprendre l’origine des doubles appels API observés au chargement.
- Optimiser le polling des notifications sans dégrader la fraîcheur des données.
- Réduire le poids CSS et éviter les traitements ou rendus inutiles.
- Garantir la compatibilité du thème sombre et le respect WCAG AA.
- Mesurer les gains sur des conditions de test reproductibles.

Organisationelles

- Coordonner les interventions frontend, backend, UX/UI et tests.
- Intégrer les optimisations dans la roadmap de six mois.
- Définir des critères d'acceptation et des indicateurs communs.
- Prévoir des mesures avant/après pour chaque évolution.
- Éviter les régressions fonctionnelles pendant les optimisations.

Budgetaires

- Disponibilité du temps de développement et de recette.
- Coût potentiel des campagnes de mesure et de suivi.
- Arbitrage entre optimisations prioritaires et évolutions fonctionnelles.
- Risque de surinvestissement sur des optimisations dont le bénéfice reste à démontrer.
- Besoin éventuel d'outils de profilage ou de suivi complémentaires.

Humaines

- Sensibilisation de l'équipe aux principes d'écoconception.
- Disponibilité des développeurs pour analyser et optimiser l'existant.
- Appropriation des nouveaux indicateurs environnementaux.
- Maintien de l'accessibilité et du confort d'utilisation.
- Risque de résistance au changement ou de manque de connaissances spécifiques.

Cartographie des contraintes projet - Détails

Domaine	Contrainte / risque identifié	Conséquence possible	Action préventive
Technique	Doubles appels API au chargement	Consommation réseau inutile	Identifier les initiateurs et corriger les appels redondants
Technique	Polling fréquent des notifications	Sollicitations répétées du serveur	Adapter la fréquence et suspendre le polling hors écran
Technique	Optimisations CSS et React	Régressions d'affichage	Tester chaque évolution et comparer avant/après
Technique	Thème sombre	Contrastes insuffisants	Vérifier les critères WCAG AA
Organisation	Dépendances frontend/backend	Retards et blocages	Planifier les interventions et les validations
Organisation	Multiplicité des indicateurs	Difficulté à suivre les résultats	Centraliser les KPI et définir un protocole de mesure
Budget	Temps de développement limité	Réduction du périmètre	Prioriser selon impact environnemental et effort
Budget	Gains difficiles à quantifier	Investissements mal orientés	Mesurer avant de généraliser une optimisation
Humain	Manque de sensibilisation	Faible adoption des pratiques	Partager les résultats et les bonnes pratiques
Humain	Changements d'interface	Inconfort ou incompréhension utilisateur	Associer les utilisateurs aux tests
Humain	Nouvelles méthodes de mesure	Difficulté d'appropriation	Prévoir une courte montée en compétence

Cartographie des contraintes projet - Priorisation

🔴 Priorité 1
À traiter dès le début du projet

- Comprendre les doubles appels API.
- Maîtriser les sollicitations liées au polling des notifications.
- Organiser les mesures initiales et définir un protocole de comparaison.
- Identifier les dépendances entre frontend et backend.

🟡 Priorité 2
À anticiper pendant les développements

- Prévenir les régressions lors des optimisations CSS et React.
- Garantir l'accessibilité du thème sombre.
- Préserver les performances lors du traitement de grandes listes.
- Réserver du temps pour les tests et les validations.

🟢 Priorité 3
À accompagner tout au long du projet

- Sensibiliser les équipes à l'écoconception.
- Suivre les coûts et la charge de travail.
- Faciliter l'adoption des nouveaux indicateurs.
- Partager les résultats et valoriser les gains obtenus.

Cartographie des contraintes projet - Challenges

- La mise en œuvre de la démarche d'écoconception du portail repose sur la prise en compte de quatre dimensions complémentaires : **technique, organisationnelle, budgétaire et humaine**.

- Les premières analyses mettent en évidence **un enjeu prioritaire autour de la maîtrise des échanges réseau, notamment les appels API redondants et le rafraîchissement fréquent des notifications.**

- La réussite du projet et le respect de la roadmap des 6 mois dépendra également de l**a coordination des équipes, de la disponibilité des ressources, de la maîtrise des coûts et de l'appropriation des bonnes pratiques d'écoconception.**

- L'objectif est donc de **mettre en place une démarche progressive, mesurable et collaborative,** permettant de **réduire l'empreinte environnementale du portail tout en préservant ses performances, son accessibilité et la qualité de l'expérience utilisateur.**